SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code: CSA10 / CSA1005	Course Title: Software Engineering
CO Assessed: CO1	Assessment: Assessment Tool 1 – Case Study Analysis
Weightage: 20%	Total Marks: 25
Student Name: **A Mohammad Naheed**	Reg. No.: 192525336
Date: _____	Duration: 60 minutes

CO1 Statement: Apply advanced methodologies and agile project management techniques to manage software projects effectively.

Code of Conduct

I A Mohammad Naheed/192525336 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A Mohammad Naheed

Annexure A – Case Study Analysis

Problem Statement No. 33: Smart Courier and Last-Mile Delivery Management Platform

1. Understand the Problem Statement

1.1 The Problem in My Own Words

E-commerce and logistics companies deliver very large numbers of parcels every day to homes and offices in cities, towns and villages. The last mile, which is the final stretch from the local hub to the customer's door, is the most expensive and least predictable part of the journey. Deliveries fail because the customer is not at home, the address is incomplete or wrong, or the agent reaches the location at a bad time. Routes are often planned by hand or by the agent's own judgement, so vehicles travel extra distance, some agents are overloaded while others are under-used, and urgent parcels are not always delivered first. Customers usually do not know when their parcel will arrive and have to call the support desk, which adds cost and frustration.

The company therefore needs one smart software platform that stores all shipment, agent, address and schedule information in one place, assigns each parcel to the most suitable agent, plans an efficient route, records every delivery attempt, handles failed deliveries properly, keeps customers informed, and produces analytics so management can keep improving the operation.

1.2 Objectives of the Proposed System

- **Manage shipment information:** create, store, update and search shipment records with weight, size, priority and delivery window.

- **Assign deliveries to agents:** allocate parcels automatically based on location, priority, vehicle capacity and working hours.
- **Manage customer addresses:** validate, standardise and geocode addresses and store customer delivery preferences.
- **Track delivery status:** show the live status of every parcel from order received to delivered or returned.
- **Plan delivery routes:** generate optimised stop sequences that reduce distance, time and fuel.
- **Record delivery attempts:** capture time, location, outcome and proof of delivery for every attempt.
- **Manage failed deliveries:** capture the reason for failure, rebook the delivery and apply the retry or return-to-sender policy.
- **Notify customers:** send SMS, e-mail and app notifications with tracking links and estimated arrival times.
- **Generate delivery analytics:** report on on-time rate, first-attempt success, cost per delivery and agent performance.

1.3 Scope of the Problem

In scope: shipment management, address management, agent and vehicle management, delivery assignment, route planning, status tracking, attempt and failed-delivery handling, customer notifications, proof of delivery, dashboards and analytics, role-based access, and integration with e-commerce order systems and map services.

Out of scope: long-haul and inter-city transportation, warehouse inventory management, payment gateway development, and vehicle manufacturing or maintenance systems. The platform covers only the last-mile stage, from the local delivery hub to the customer, including returns that originate from failed deliveries.

2. Identify Key Stakeholders and Challenges

2.1 Stakeholders, Roles and Responsibilities

Stakeholder	Role and Responsibilities	Major Challenges Faced
Customer / Receiver	Provides correct address and contact number, chooses a delivery slot, receives the parcel, gives OTP or signature, rates the service.	No visibility of delivery time, missed deliveries, repeated calls to support, no easy way to reschedule.
Delivery Agent	Collects parcels at the hub, follows the assigned route, delivers, updates status, collects proof of delivery, reports failures.	Poor routes, overloaded workload, wrong or unclear addresses, traffic, unreachable customers, manual paperwork.
Dispatcher / Hub Manager	Sorts parcels, assigns agents and vehicles, monitors progress, handles exceptions and re-assignments.	Manual and time-consuming assignment, no real-time view of agents, last-minute changes and cancellations.

Stakeholder	Role and Responsibilities	Major Challenges Faced
Logistics Operations Manager	Defines delivery policies, service levels and retry rules; monitors performance and costs.	Rising cost per delivery, lack of reliable data, difficulty in balancing cost and service quality.
Customer Support Team	Answers queries, resolves complaints, reschedules failed deliveries.	High call volume for "where is my parcel", scattered information across systems.
E-commerce Seller / Retail Partner	Sends shipment orders, defines priority and delivery promise, receives status and returns.	Customer complaints, returns caused by failed deliveries, limited visibility of delivery status.
System Administrator / IT Team	Maintains the platform, manages users, security, backups and integrations.	System downtime, integration with legacy systems, data security, scaling during festival and sale seasons.
Management / Business Owner	Funds the project, sets business goals, uses analytics for decisions.	High operational cost, unclear return on investment, loss of customers because of delays.
Regulatory and Third-Party Services	Map and GPS providers, SMS gateways, data-protection authorities.	Service dependence, API limits and cost, compliance with privacy rules.

2.2 Major Challenges and Issues

- **Failed deliveries:** customer unavailable, wrong address, refused parcel or unreachable phone number, leading to repeated attempts and return costs.
- **Inefficient routes:** manual sequencing wastes fuel and time and causes delays.
- **Address problems:** incomplete, informal or duplicate addresses, especially in rural and fast-growing urban areas.
- **Capacity and constraint balancing:** assigning parcels while respecting vehicle weight and volume limits, shift hours, priorities and delivery time windows.
- **Lack of real-time communication:** customers, agents and dispatchers do not share one live view of the delivery.
- **Demand peaks:** sudden volume increases during sales and festivals strain agents and systems.
- **Poor data and analytics:** records kept in spreadsheets or separate tools make it hard to find the causes of delay.

3. Analyze the Current Approach

3.1 Existing System or Current Process

In many courier companies the present process is largely manual or semi-automated. Orders arrive from e-commerce partners as files or through basic integrations. At the hub, the dispatcher groups parcels by area using personal experience or spreadsheets and hands them to agents. The agent decides the visiting order, calls the customer by phone when the address is unclear, and marks the parcel delivered on a paper slip or a simple app. If delivery fails, the agent informs the hub verbally or at the

end of the day, and the parcel is tried again the next day or returned after a fixed number of attempts. Customers receive limited SMS messages or have to call support to know the parcel status. Reports are prepared manually at the end of the week or month.

3.2 Strengths and Limitations

Strengths of the Current Process	Limitations of the Current Process
Simple and easy for staff to understand; little training needed.	No automatic optimisation of agent allocation or routes.
Experienced agents know local roads and customers well.	Local knowledge is not recorded and is lost when agents leave.
Low initial technology cost.	High running cost through fuel, overtime and repeated attempts.
Flexible; humans can handle unusual cases by phone.	Slow and error-prone manual data entry and status updates.
Works with existing partners without major system changes.	No real-time tracking or proactive customer notifications.
	Poor address quality control; failed-delivery reasons are not analysed.
	Weak scalability during peak seasons; limited reporting and analytics.

3.3 Problems and Gaps to be Addressed

- No intelligent, rule-based assignment using location, priority, capacity and time windows.
- No route optimisation or dynamic re-routing when traffic, cancellations or urgent orders occur.
- No central, validated address repository with geocoding and customer delivery preferences.
- No structured recording of delivery attempts, failure reasons, and retry or return rules.
- No live visibility for customers, support staff and managers.
- No data-driven analytics for decisions on cost, performance and capacity planning.
- No proof of delivery that is digital, secure and linked to the shipment record.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Proposed Software-Based Solution

The proposed solution is a cloud-based Smart Courier and Last-Mile Delivery Management Platform made up of three main interfaces: a web portal for dispatchers, managers and support staff; a mobile application for delivery agents; and a customer tracking portal and mobile app. All three connect to a central back-end that uses a layered, service-oriented architecture (presentation layer, business-logic services, data layer) and integrates with map and routing services, an SMS and e-mail gateway, and the e-commerce order systems through REST APIs. A relational database holds shipment, address, agent and delivery-attempt data, and a caching and queue layer supports real-time updates and notifications.

When a shipment arrives, the platform validates and geocodes the address, calculates priority, assigns the shipment to the best agent and vehicle, includes it in an optimised route, and notifies the customer with an estimated time window. The agent follows the route in the mobile app, updates the status at each stop, and records proof of delivery. If delivery fails, the agent selects a reason; the system notifies the customer, offers rescheduling, and applies the retry or return policy automatically. All events feed the analytics module.

4.2 Major Functional Modules

Module	Description
1. Shipment Management	Create, import (bulk or API), edit, search and cancel shipments; store weight, volume, priority, service type and delivery window.
2. Address Management	Capture, validate, standardise and geocode addresses; store landmarks, preferred time slots and alternate contacts; detect duplicates.
3. Agent and Vehicle Management	Maintain agent profiles, shifts, availability, zones, vehicle type and capacity.
4. Delivery Assignment Engine	Allocate shipments to agents based on location clusters, priority, capacity and working-hour constraints; allow manual override.
5. Route Planning and Optimisation	Produce an efficient stop sequence using map data and time windows; re-optimise dynamically for changes.
6. Delivery Status Tracking	Maintain status life-cycle (Created, Assigned, Out for Delivery, Delivered, Failed, Rescheduled, Returned) with live agent location.
7. Delivery Attempt Recording	Log each attempt with time, GPS location, outcome, photo, OTP or signature as proof of delivery.
8. Failed Delivery Management	Capture reason codes, trigger customer contact, reschedule, change address or pick-up point, and return-to-sender workflow.
9. Customer Notification	Send SMS, e-mail and push notifications for dispatch, ETA, arrival, delivered and failed events; link to tracking page.
10. Analytics and Reporting	Dashboards and reports on on-time rate, first-attempt success, failure causes, route utilisation, cost per delivery and agent productivity.
11. User and Access Management	Authentication, role-based authorisation, audit logs and configuration of policies.

4.3 Functional Requirements

- **FR1:** The system shall allow authorised users to create, update, search and cancel shipments individually or in bulk.
- **FR2:** The system shall validate and geocode customer addresses and store multiple addresses per customer.
- **FR3:** The system shall assign shipments to agents using location, priority, vehicle capacity and shift constraints.
- **FR4:** The system shall generate and display an optimised route for each agent and re-plan when conditions change.

- **FR5:** The system shall let agents update delivery status and record proof of delivery from the mobile app, including offline use.
- **FR6:** The system shall record every delivery attempt with time, location and outcome.
- **FR7:** The system shall capture a failure reason and automatically trigger rescheduling or return according to policy.
- **FR8:** The system shall notify customers at key events and let them reschedule or change delivery instructions.
- **FR9:** The system shall provide real-time tracking for customers and dispatchers.
- **FR10:** The system shall generate delivery analytics, dashboards and exportable reports.
- **FR11:** The system shall provide role-based access for admin, dispatcher, agent, support and customer.

4.4 Non-Functional Requirements

Attribute	Requirement
Performance	Pages and APIs respond within 2 seconds for normal load; route optimisation for one agent completes within 10 seconds.
Scalability	Support growth to several hundred thousand shipments per day and peak-season surges through horizontal scaling.
Reliability / Availability	99.5% or higher uptime; automatic failover, backups and retry of failed notifications.
Security	Encrypted communication (HTTPS/TLS), encrypted storage of personal data, role-based access, secure OTP, audit trails.
Usability	Simple, multi-language mobile app with few taps per delivery; accessible customer tracking page.
Maintainability	Modular code, documented APIs, automated tests and easy configuration of business rules.
Interoperability	Standard REST/JSON APIs for e-commerce, map, SMS and payment integrations.
Portability and Compatibility	Works on Android, iOS and major web browsers; cloud-vendor neutral deployment using containers.
Compliance	Follow applicable data-protection and privacy regulations for customer data.

4.5 Application of Software Engineering Principles

- **Modularity:** The system is divided into independent modules or services (shipment, address, assignment, routing, notification, analytics) with clear interfaces, so each can be developed, tested and replaced separately.
- **Scalability:** Stateless services, containerisation, load balancing, message queues and database indexing allow the platform to scale horizontally during peak seasons.
- **Maintainability:** Clean layered architecture, coding standards, version control, code reviews, automated unit and integration tests, and configurable business rules reduce the cost of change.

- **Reliability:** Redundant servers, database backups, retry mechanisms, offline mode in the agent app and monitoring with alerts keep the service running even when a component fails.
- **Usability:** User-centred design with role-specific screens, minimal data entry for agents, simple tracking links for customers and usability testing with real users.
- **Security:** Authentication, role-based authorisation, encryption of data in transit and at rest, input validation, audit logging and regular vulnerability testing protect customer and operational data.
- **Abstraction and Reusability:** Map, SMS and routing providers are accessed through adapter interfaces so providers can be changed without affecting core logic.

5. Justify the Chosen Methodology / Framework

5.1 Selected Framework

Agile with the Scrum framework, supported by DevOps practices (continuous integration and continuous delivery), is selected for developing and maintaining the platform.

5.2 Justification

- **Evolving requirements:** Delivery rules, address formats, partner integrations and customer expectations change often. Scrum accepts changing requirements at the start of every sprint, whereas Waterfall would find it costly to change.
- **Early and frequent value:** Core features such as shipment management, assignment and status tracking can be released in the first sprints, and route optimisation, notifications and analytics follow in later increments.
- **Continuous stakeholder feedback:** Dispatchers, agents and customer-support staff review each sprint demo, so the product matches real field conditions.
- **Risk reduction:** Complex parts such as route optimisation and map integration are built and tested in small increments, so problems are found early. A Spiral-style risk review is used before each release.
- **Prioritised backlog:** The product owner ranks work by business value, such as reducing failed deliveries first.
- **Team collaboration and transparency:** Daily stand-ups, sprint planning, review and retrospective meetings keep the team aligned and make progress visible.
- **Suitable for a multi-interface product:** Web portal, agent app and customer app can be developed in parallel by cross-functional teams.

5.3 How It Supports Successful Development and Maintenance

- Product backlog and user stories translate business needs into small, testable work items; sprints of two to three weeks give a regular release rhythm.
- Definition of Done, automated testing and CI/CD pipelines keep quality high and make releases quick and repeatable.
- Sprint reviews and retrospectives allow continuous process improvement.
- After the first release, the same backlog-driven cycle handles bug fixes, enhancements and seasonal changes, so maintenance is part of normal development rather than a separate phase.

- Burndown charts, velocity and the sprint board help in tracking progress, estimating effort and managing the project effectively, which directly supports CO1.

5.4 Comparison with Other Models

Model	Reason for Not Choosing as the Primary Model
Waterfall	Fixed requirements and late testing; unsuitable for a platform whose rules and integrations keep changing.
Spiral	Good for risk handling but heavy on documentation and cost for a commercial product; its risk analysis idea is used within Scrum.
DevOps only	Focuses on delivery and operations but needs a planning framework such as Scrum; therefore it is used together with Scrum.

6. Discuss Expected Outcomes and Limitations

6.1 Expected Benefits

- **Improved delivery efficiency:** automated assignment and routing reduce planning time and idle time.
- **Reduced failed delivery attempts:** validated addresses, preferred time slots and proactive notifications increase first-attempt success.
- **Better route utilisation:** optimised routes cut distance travelled and make better use of vehicle capacity.
- **Improved customer visibility:** live tracking and estimated arrival times reduce support calls and raise satisfaction.
- **Reduced delivery delays:** priority handling and dynamic re-routing keep urgent parcels on time.
- **Lower operational costs:** less fuel, fewer repeat attempts, fewer returns and fewer manual tasks.

6.2 How the Solution Addresses the Identified Challenges

Challenge	How the Proposed Solution Addresses It
Failed deliveries	Address validation, slot selection, customer alerts, structured failure reasons and automatic rescheduling.
Inefficient routes	Route planning and optimisation module with dynamic re-routing.
Address problems	Central address repository with geocoding, landmarks and duplicate detection.
Capacity and priority constraints	Assignment engine that applies capacity, priority, time-window and shift rules.
Lack of real-time communication	Live tracking, push/SMS/e-mail notifications and shared dashboards.
Poor data and reporting	Analytics module with dashboards and exportable reports.

Challenge	How the Proposed Solution Addresses It
Peak-season load	Scalable cloud architecture with queues and load balancing.

6.3 Potential Risks, Implementation Challenges and Limitations

- **Data quality risk:** poor or missing address data in rural areas may still lower geocoding accuracy.
- **Connectivity:** weak mobile network in remote areas may delay live updates; the offline mode only partly reduces this.
- **User adoption:** agents and staff may resist change; training and incentives are needed.
- **Integration complexity:** legacy e-commerce and partner systems may have inconsistent data formats.
- **Third-party dependency:** map, routing and SMS providers can fail, change prices or limit API usage.
- **Optimisation limits:** route and assignment problems are computationally hard, so the solution gives near-optimal rather than perfect results and cannot predict traffic accidents or weather.
- **Security and privacy:** personal data and location tracking require strict protection and compliance.
- **Cost and schedule:** initial development, cloud and device costs, and scope creep must be controlled.
- **Human factors:** customer unavailability or refusal cannot be removed completely.

6.4 Possible Future Enhancements

- Machine-learning models to predict delivery time, failure probability and demand peaks.
- Customer self-service: live agent tracking on map, in-app chat or call masking.
- Smart lockers, pick-up points and alternative delivery locations.
- Support for electric vehicles, charging-aware routing and carbon-footprint reporting.
- Integration of drones, delivery robots or crowd-sourced gig agents.
- Voice-based and multilingual address capture, and what3words-style digital addressing.
- IoT sensors for parcel condition monitoring (temperature, shock) for sensitive items.
- Blockchain-based or biometric proof of delivery for high-value parcels.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology / Framework Justification; Outcomes, limitations, future enhancements (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5

Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	**/25**